

> [MC613 - Laboratório de Circuitos Digitais](#) > Projeto: Controlador DRAM

Projeto: Controlador DRAM

Nos projetos anteriores, exploramos o desenvolvimento de sistemas digitais a partir de especificações definidas diretamente no enunciado. Neste projeto, avançamos um passo além: trabalharemos com um componente real da placa, cuja interface e comportamento são descritos em um [datasheet externo](#).

A memória SDRAM presente na DE1-SoC é um exemplo de sistema digital que exige controle preciso de comandos e temporização. Diferente de periféricos simples, seu funcionamento depende de uma sequência estruturada de operações, respeitando restrições temporais e protocolos bem definidos.

O objetivo deste projeto é desenvolver um **controlador de memória DRAM simplificado**, capaz de realizar operações básicas de leitura, escrita e refresh. Para isso, será necessário interpretar a documentação do componente, identificar as informações relevantes e traduzir esse comportamento em um sistema digital implementado em VHDL.

► [Ver histórico de mudanças](#)

Disciplina

Este projeto faz parte da disciplina MC613 - Laboratório de Circuitos Digitais. [Ver oferecimento mais recente](#).

Recursos necessários

- [Manual da placa DE1-SoC](#)
- [Datasheet do chip SDRAM](#).

Como funciona uma memória DRAM

A memória DRAM (Dynamic Random Access Memory) é amplamente utilizada em sistemas computacionais como memória principal. Diferente de memórias mais simples, seu funcionamento envolve **múltiplas etapas, comandos específicos e restrições de tempo** que devem ser respeitadas para garantir operação correta.

Esta seção apresenta os conceitos fundamentais necessários para compreender e projetar um controlador de DRAM.

Comandos e sequência de operações

A DRAM é controlada por uma sequência de comandos. Os principais comandos utilizados neste projeto são:

- **ACTIVATE**: abre uma linha da memória;
- **READ**: lê um dado de uma coluna da linha aberta;
- **WRITE**: escreve um dado em uma coluna da linha aberta;
- **PRECHARGE**: fecha a linha ativa;
- **REFRESH**: reestabelece os dados armazenados nas células;
- **LOAD MODE REGISTER**: configura parâmetros internos da memória.

Cada operação de leitura ou escrita envolve uma sequência ordenada desses comandos.

Os comandos descritos aqui são representações lógicas. No hardware, eles são implementados por combinações de sinais de controle. Os sinais enviados à DRAM podem ser divididos em dois grupos:

- **Sinais de controle (comandos)**: indicam qual operação deve ser realizada;
- **Sinais de dados**: transportam os valores a serem escritos ou lidos.

Durante uma operação:

- O controlador envia comandos em ciclos específicos;
- Os dados são fornecidos (WRITE) ou capturados (READ) em momentos definidos pela temporização.

ACTIVATE e READ/WRITE

Para entender como esses comandos funcionam, é necessário compreender como a memória está organizada internamente. A memória DRAM é organizada hierarquicamente em **bancos (banks)**, e cada banco contém uma matriz bidimensional de **linhas (rows)** e **colunas (columns)**. O endereço completo de memória é dividido em partes que selecionam o banco, a linha e a coluna.

Os comandos enviados à memória (como ACTIVATE, READ e WRITE) são sempre direcionados a um banco específico.

Para acessar um dado, o processo ocorre em duas etapas dentro de um banco:

1. **Seleção da linha (ACTIVATE)**
2. **Seleção da coluna (READ ou WRITE)**

Quando uma linha é ativada em um banco, **todos os dados dessa linha são carregados para um buffer interno**, chamado de *row buffer*, associado àquele banco.

A partir disso:

- operações de leitura e escrita acessam colunas específicas da linha ativa;
- cada banco possui seu próprio *row buffer* independente.

Isso significa que diferentes bancos podem manter linhas distintas abertas simultaneamente, de forma independente.

PRECHARGE

Após o comando de ativação (ACTIVATE), a linha permanece aberta no *row buffer*.

Enquanto a linha estiver aberta:

- É possível acessar diferentes colunas dessa mesma linha;
- Não é possível acessar outra linha.

Para acessar uma nova linha, é necessário primeiro **fechar a linha atual**, utilizando o comando: **PRECHARGE**.

Esse comando descarrega o *row buffer*, preparando a memória para um novo ciclo de acesso.

Ou seja, o comando PRECHARGE encerra o acesso iniciado por um ACTIVATE, fechando a linha ativa.

REFRESH

A DRAM armazena dados em capacitores, que naturalmente perdem carga ao longo do tempo. Para evitar perda de dados, é necessário realizar periodicamente uma operação de **REFRESH**

Essa operação reescreve os dados armazenados, restaurando sua integridade.

No funcionamento do sistema:

- O refresh deve ocorrer periodicamente;
- Durante o refresh, a memória não pode ser acessada para leitura ou escrita.
- O controlador deve interromper ou adiar operações de leitura/escrita para executar o refresh no momento adequado.

Inicialização da Memória (e LOAD MODE REGISTER)

Após energizar o sistema, a DRAM não está pronta para uso imediato. É necessário executar uma **sequência de inicialização**, que:

- Prepara a memória para operação normal;
- Configura parâmetros internos (como latência e modo de operação);
- Garante que o estado inicial da memória seja válido.

Essa sequência envolve comandos específicos definidos no datasheet e deve ser executada antes de qualquer operação de leitura ou escrita.

Temporização e latência

Diferente de memórias ideais, a DRAM possui **restrições de tempo entre comandos**, definidas no datasheet. Essas restrições existem devido ao funcionamento físico da memória (carga e descarga de capacitores, amplificação de sinais, etc.).

Alguns parâmetros importantes incluem:

- **tRCD (Row to Column Delay)**: tempo entre ACTIVATE e READ/WRITE;
- **CAS Latency (CL)**: tempo entre o comando READ e a disponibilidade do dado;
- **tRP (Row Precharge Time)**: tempo necessário após PRECHARGE antes de um novo ACTIVATE;

- **tWR (Write Recovery Time)**: tempo necessário após uma escrita antes de fechar a linha;
- **tRFC (Refresh Cycle Time)**: duração de um ciclo de refresh.

Cabe ao controlador garantir que os intervalos mínimos entre comandos sejam respeitados. Isso pode ser implementado como **esperas** (em ciclos de clock) até a próxima operação.

Leitura: latência e captura de dados

Após o envio de um comando **READ**, o dado não é disponibilizado imediatamente.

Existe um atraso, definido pela **CAS Latency**, até que o valor esteja válido no barramento de dados. O controlador deve aguardar esse tempo antes de capturar o valor lido.

Escrita: envio de dados

Durante uma operação de **WRITE**, o controlador deve fornecer o dado no momento correto após o comando de escrita. Além disso, deve respeitar um tempo mínimo (**tWR**) antes de encerrar a operação com um PRECHARGE.

Modelo de operação adotado neste projeto

Neste projeto, o controlador será desenvolvido com as seguintes simplificações:

- Apenas **uma operação por vez** (sem paralelismo ou pipeline);
- Cada acesso envolve a sequência completa:
 - **ACTIVATE → READ/WRITE → PRECHARGE**;
- Não serão utilizadas otimizações como múltiplas linhas abertas;
- O comportamento será controlado por uma máquina de estados sequencial.

Observação importante:

Neste projeto, o comportamento correto depende não apenas dos comandos enviados, mas também do **tempo entre eles**.

Nem todos os efeitos são visíveis diretamente na placa. Por isso, a validação do sistema deve considerar:

- A sequência de comandos gerados;

- Os tempos entre comandos;
- Os dados lidos e escritos.

A simulação será uma ferramenta essencial para verificar o funcionamento correto do controlador.

Visão geral do sistema

Para organizar o desenvolvimento do projeto, o sistema foi dividido em dois módulos principais:

- ``dram_iface``
- ``dram_controller``

Essa separação permite isolar responsabilidades e reduzir a complexidade de cada parte do sistema.

O módulo ``dram_iface`` é responsável pela **interação com o usuário**. Ele recebe os valores dos switches e botões da placa, interpreta essas entradas e gera solicitações de leitura e escrita. Além disso, implementa uma máquina de estados simples que controla quando essas operações devem ser realizadas e exibe os resultados nos displays.

O módulo ``dram_controller`` é responsável por **controlar diretamente a memória DRAM**. Ele recebe comandos de leitura e escrita vindos do ``dram_iface`` e os traduz em sequências de comandos e sinais compatíveis com o protocolo da memória, respeitando as restrições de temporização descritas no datasheet.

Por fim, um módulo de topo (``top_level``) conecta esses dois blocos entre si e realiza a interface com os pinos físicos da placa.

Essa divisão reflete a organização típica de sistemas reais, nos quais uma camada de alto nível gerencia requisições e uma camada de baixo nível implementa o protocolo do hardware.

``dram_iface``: Interface com o usuário

O módulo ``dram_iface`` é responsável por realizar a interface entre o usuário (por meio dos switches, botões e displays da placa) e o controlador de memória DRAM (``dram_controller``).

Seu papel é:

- Interpretar as entradas do usuário (switches e botões);
- Converter essas entradas em comandos e sinais compatíveis com o controlador;
- Gerenciar uma máquina de estados simples que organiza as operações de leitura e escrita;
- Exibir informações relevantes nos displays da placa.

Este módulo **não implementa o protocolo da DRAM**. Ele atua como uma camada de controle de alto nível, organizando quando e como as operações devem ser solicitadas ao ``dram_controller``.

Descrição operacional

Do ponto de vista do usuário, o sistema funciona da seguinte forma:

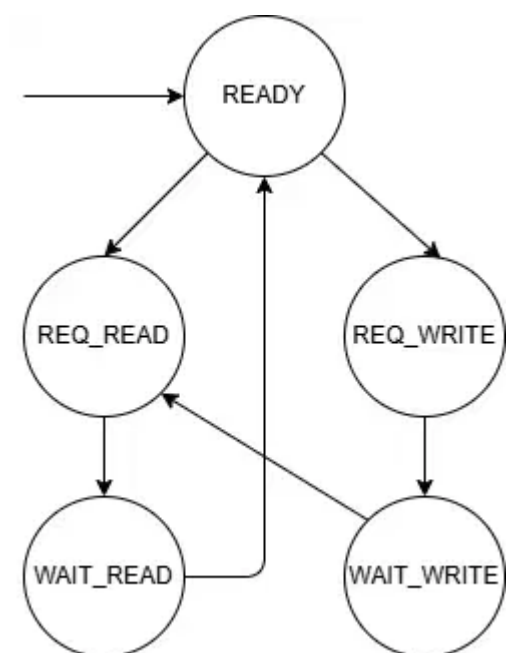
- Os switches definem:
 - Parte do endereço de memória a ser acessado;
 - O valor de dado a ser escrito;
- Sempre que o valor dos switches correspondente ao endereço muda:
 - Uma operação de **leitura** é automaticamente solicitada;
 - O valor lido da memória é exibido nos displays;
- Quando o botão ``KEY[3]`` é pressionado:
 - Uma operação de **escrita** é solicitada com o valor definido nos switches;
 - Após a escrita, uma leitura é automaticamente realizada para confirmar o valor armazenado;
- O sistema processa uma operação por vez. Novas solicitações só são aceitas quando o controlador estiver pronto.

Entradas e saídas

Sinal	Direção	Largura	Descrição
clk	Entrada	1 bit	Clock

Sinal	Direção	Largura	Descrição
rst	Entrada	1 bit	Reset
SW	Entrada	10 bits	Switches de entrada (endereço parcial e dado)
KEY	Entrada	4 bits	Botões da placa (`KEY[3]` = write, `KEY[0]` = reset)
HEX0	Saída	7 bits	Exibe o dado de entrada (escrita)
HEX1	Saída	7 bits	Exibe o valor lido da memória
HEX4	Saída	7 bits	Exibe parte do endereço
HEX5	Saída	7 bits	Exibe parte do endereço
address	Saída	26 bits	Endereço completo da DRAM
data	Entrada/Saída	8 bits	Dado lido/a ser escrito
req	Saída	1 bit	Indica a emissão de um comando para o controlador.
wEn	Saída	1 bit	Indica sinal de permissão de escrita (o comando é uma escrita)
ready	Entrada	1 bit	Indica que o controlador está pronto para receber uma nova operação

Estados da máquina de controle



O módulo ``dram_iface`` é controlado por uma máquina de estados finitos (FSM), responsável por coordenar as operações de leitura e escrita.

``READY``: Estado ocioso do sistema, no qual o módulo aguarda novas solicitações do usuário. Neste estado, o ``dram_iface`` monitora continuamente os switches e o botão ``KEY[3]``, além do sinal ``ready`` do controlador. Caso o controlador esteja pronto (``ready = 1``), duas transições são possíveis: se o valor do endereço (derivado dos switches) for diferente do último valor utilizado, o sistema inicia uma leitura e transita para ``REQ_READ``; se o botão ``KEY[3]`` for pressionado, o sistema inicia uma escrita e transita para ``REQ_WRITE``. Caso contrário, permanece em ``READY``.

``REQ_READ``: Estado de solicitação de leitura, no qual o endereço atual (derivado dos switches) é enviado ao ``dram_controller`` e uma operação de leitura é iniciada. Este estado é transitório e, após a emissão da requisição, o sistema avança imediatamente para o estado ``WAIT_READ``.

``WAIT_READ``: Estado de espera pela conclusão da leitura. O módulo permanece neste estado aguardando o ``dram_controller`` finalizar a operação. Durante esse período, nenhuma nova solicitação é aceita. Quando o controlador sinaliza que está pronto (``ready = 1``), o sistema considera a operação concluída e retorna ao estado ``READY``, exibindo o valor lido nos displays.

``REQ_WRITE``: Estado de solicitação de escrita, no qual o endereço e o dado (derivados dos switches) são enviados ao ``dram_controller`` e uma operação de escrita é iniciada. Assim como em ``REQ_READ``, este é um estado transitório, e o sistema avança imediatamente para ``WAIT_WRITE`` após emitir a requisição.

``WAIT_WRITE``: Estado de espera pela conclusão da escrita. O módulo aguarda o ``dram_controller`` finalizar a operação de escrita. Quando o controlador sinaliza que está pronto (``ready = 1``), o sistema automaticamente inicia uma leitura do mesmo endereço para verificar o valor armazenado, transitando para o estado ``REQ_READ``.

Conversão de endereço e dado

A memória SDRAM presente na placa possui capacidade de **64 MB**, o que corresponde a um espaço de endereçamento de **26 bits** (``address[25:0]``).

Como a placa dispõe de apenas **10 switches**, não é possível controlar diretamente todos os bits do endereço. Portanto, utilizaremos uma **subamostragem do espaço de endereçamento**,

mapeando apenas alguns bits dos switches para posições específicas do endereço, enquanto os demais bits permanecem fixos.

A conversão deve ser feita da seguinte forma:

- ``SW[9]`` → ``address[25]``
- ``SW[8:6]`` → ``address[23:20]``
- ``SW[5:4]`` → ``address[1:0]``

Todos os demais bits de ``address`` que não são controlados pelos switches devem ser **fixados em 0**.

O barramento de dados da memória possui **8 bits** (``data[7:0]``), porém utilizaremos apenas os **4 bits mais à direita** provenientes dos switches:

- ``SW[3:0]`` → ``data[3:0]``

Os demais bits devem ser tratados da seguinte forma:

- Durante **escrita**: ``data[7:4] = 0``
- Durante **leitura**: os bits ``data[7:4]`` devem ser **ignorados**

Essa simplificação permite testar o funcionamento do controlador sem exigir o controle completo dos barramentos de endereço e dados.

``dram_controller``: O controlador DRAM de fato

O módulo ``dram_controller`` é responsável por implementar o **controle direto da memória SDRAM** presente na placa. Diferente do ``dram_iface``, que atua em nível de interação com o usuário, este módulo implementa o **protocolo da DRAM**, incluindo:

- Sequenciamento de comandos (ACTIVATE, READ, WRITE, PRECHARGE, REFRESH);
- Respeito às restrições de temporização;
- Inicialização da memória;
- Controle do fluxo de dados de leitura e escrita.

Este é o núcleo do projeto, no qual o comportamento descrito no datasheet deve ser traduzido em uma **máquina de estados finitos (FSM)**.

Descrição operacional

O ``dram_controller`` recebe requisições de leitura e escrita e as converte em sequências de comandos compatíveis com a DRAM.

O funcionamento geral segue o fluxo:

1. Após o reset, o controlador executa a sequência de **inicialização da memória**;
2. Entra no estado ``READY``, onde pode receber novas requisições;
3. Ao receber uma requisição:
 - Executa uma sequência de comandos correspondente à operação (READ ou WRITE);
 - Respeita os tempos mínimos entre comandos;
4. Periodicamente, executa operações de **REFRESH**, interrompendo o fluxo normal se necessário;
5. Retorna ao estado ``READY`` ao final de cada operação.

Entradas e saídas

Sinal	Direção	Largura	Descrição
clk	Entrada	1 bit	Clock
rst	Entrada	1 bit	Reset
address	Entrada	26 bits	Endereço completo da DRAM
data	Entrada/Saída	8 bits	Dado lido/a ser escrito
req	Entrada	1 bit	Indica a recepção de um comando no controlador.
wEn	Entrada	1 bit	Indica sinal de permissão de escrita (o comando é uma escrita)
ready	Saída	1 bit	Indica que o controlador está pronto para receber uma nova operação

******A tabela acima demonstra as entradas e saídas para interface com o ``dram_iface``. Além delas, devem ser consideradas as entradas e saídas para interface com o módulo DRAM (via top-level), conforme [Manual da Placa](#) e [datasheet](#).

Estados da máquina de controle

A máquina de estados do ``dram_controller`` deve ser organizada em um nível alto com os seguintes estados principais:

``INIT``: Estado responsável pela inicialização da memória DRAM.

Neste estado, o controlador executa a sequência de inicialização descrita no datasheet, incluindo:

- Comandos de precharge;
- Comandos de refresh;
- Configuração do mode register.

Após a conclusão, o controlador transita para o estado ``READY``.

``READY``: Estado ocioso no qual o controlador está disponível para receber requisições.

- A saída ``ready`` deve estar em ``1``;
- O controlador aguarda:
 - Requisições de leitura/escrita (``req``);
 - Necessidade de refresh (baseado em temporização interna).

Transições:

- Para ``READ`` ou ``WRITE``, conforme o comando recebido;
- Para ``REFRESH``, quando necessário.

``READ``: Estado responsável pela execução de uma operação de leitura.

A implementação deste estado deve seguir a sequência:

1. ACTIVATE (seleção da linha);
2. Espera do tempo tRCD;
3. READ (com endereço de coluna);
4. Espera da latência CAS;
5. Captura do dado;
6. PRECHARGE (fechamento da linha);
7. Espera do tempo tRP.

Após a conclusão, retorna ao estado **`READY`**.

`WRITE`: Estado responsável pela execução de uma operação de escrita.

A sequência esperada é:

1. ACTIVATE;
2. Espera do tempo t_{RCD} ;
3. WRITE;
4. Envio do dado;
5. Espera do tempo t_{DPL} ;
6. PRECHARGE;
7. Espera do tempo t_{RP} .

Após a conclusão, retorna ao estado **`READY`**.

`REFRESH`: Estado responsável pela execução de um ciclo de refresh.

A sequência consiste em:

1. Comando AUTO REFRESH;
2. Espera do tempo t_{RC} .

Após a conclusão, retorna ao estado **`READY`**.

Importante:

Os estados acima representam uma visão de alto nível.

Cabe ao grupo refinar essa máquina de estados, introduzindo estados intermediários conforme necessário para implementar corretamente:

- Cada comando;
- Os tempos de espera;
- A sequência exigida pelo datasheet.

Configuração e restrições do projeto

Nesta seção são definidas as configurações específicas que devem ser utilizadas no projeto.

Dispositivo utilizado:

- Modelo: **IS42S16320D-7TL**
- Clock base: **143 MHz** (ver tabela na página 1 do datasheet)

Configuração do Mode Register (página 26):

Utilizar os seguintes parâmetros:

- Burst Length = 1
- Burst Type = Sequential
- CAS Latency = 2
- Operating Mode = Standard
- Write Burst Mode = Single Location Access

Inicialização:

Seguir o ciclo de inicialização descrito na página 23 do datasheet.

Refresh:

Utilizar o modo **CBR (Auto Refresh)**.

Referência: página 24 do datasheet.

Operações de leitura e escrita:

Utilizar os seguintes modelos:

- Single Read Without Auto Precharge (página 56)
- Single Write Without Auto Precharge (página 60)

Esses modelos definem a sequência de comandos e temporização que devem ser implementadas.

Roteiro de planejamento

O desenvolvimento deste projeto depende diretamente da capacidade de interpretar corretamente o datasheet da memória SDRAM.

O datasheet **não deve ser lido de forma linear**. Em vez disso, utilize o roteiro abaixo para identificar as informações necessárias para o projeto.

Interface com a memória

- Localize no datasheet a descrição dos sinais da memória - página 7;
- Identifique:
 - sinais de comando (CS, RAS, CAS, WE);
 - sinais de endereço;
 - sinais de banco;
 - sinais de dados;
- Relacione esses sinais com os pinos disponíveis na FPGA (manual da placa).

Objetivo: definir as entradas e saídas do ``dram_controller`` e ``top_level``.

Codificação de comandos

- Localize a tabela de comandos (command truth table) - página 9;
- Identifique como são codificados os comandos:
 - ACTIVATE
 - READ
 - WRITE
 - PRECHARGE
 - AUTO REFRESH
- Observe quais sinais precisam ser ativados/desativados para cada comando.

Objetivo: entender como gerar cada comando no controlador.

Sequência de operações

- Consulte os diagramas de leitura e escrita:
 - *Single Read Without Auto Precharge* (página 56)
 - *Single Write Without Auto Precharge* (página 60)

- Identifique:
 - ordem dos comandos;
 - quando os dados são válidos;
 - quando cada comando pode ser emitido.

Objetivo: definir a sequência de estados da máquina de controle.

Parâmetros de temporização

- Localize a tabela de parâmetros de tempo (timing parameters, páginas 19 e 20);
- Identifique os principais tempos:
 - tRCD (ACTIVATE -> READ/WRITE)
 - tCAS (latência de leitura)
 - tRP (PRECHARGE)
 - tRC e tREF (REFRESH)
 - tDPL (WRITE -> PRECHARGE)

Objetivo: determinar quantos ciclos de clock devem ser aguardados entre comandos.

Inicialização da memória

- Consulte o ciclo de inicialização - página 23;
- Identifique a sequência de comandos necessária após o reset;
- Verifique a configuração do mode register.

Objetivo: definir o comportamento do estado **INIT**.

6. Refresh

- Consulte a descrição do comando AUTO REFRESH e o diagrama do ciclo de refresh - páginas 8 e 24;
- Identifique:
 - quando deve ocorrer;
 - quanto tempo deve ser aguardado.

Objetivo: definir o comportamento do estado ``REFRESH``.

Resultado esperado do planejamento

Ao final deste processo, o grupo deve ser capaz de:

- Definir claramente as entradas e saídas do controlador;
- Determinar os tempos de espera entre comandos;
- Relacionar cada estado com comandos reais da DRAM;
- Especificar a máquina de estados (incluindo estados intermediários e temporização entre estados).

Este roteiro deve ser utilizado como base para a elaboração dos diagramas e planejamento do projeto.

Expectativa de entrega

O desenvolvimento do projeto será avaliado em três etapas: planejamento, checkpoint e entrega final. Cada etapa possui objetivos específicos que refletem o progresso esperado do grupo.

Planejamento (Semana 1)

Nesta etapa, o grupo deve apresentar um planejamento técnico consistente de como o projeto será implementado.

Entregáveis esperados:

- Diagrama de estados refinado do ``dram_iface``;
- Diagrama de estados refinado do ``dram_controller``, incluindo:
 - estados intermediários necessários;
 - condições de transição;
 - temporizações entre estados (em ciclos de clock);
- Explicação de quaisquer:
 - simplificações adotadas;

- adições ou refinamentos necessários em relação ao enunciado;
- ajustes nas interfaces de entrada/saída;
- Planejamento de testes por simulação para:
 - ``dram_iface``;
 - ``dram_controller``;
- Planejamento de testes na placa para:
 - ``dram_iface``;
 - sistema completo (``top_level``);

Organização da aula:

Sugere-se que este planejamento seja apresentado até a **2ª hora da aula (16h)**.

O tempo restante deve ser utilizado para iniciar a implementação.

Checkpoint (Semana 2)

Nesta etapa, o objetivo é verificar o progresso inicial da implementação, com foco na validação incremental dos módulos.

Entregáveis esperados:

- ``dram_iface``:
 - Implementado;
 - Testado por simulação;
 - Testado na placa, utilizando recursos da placa para depuração:
 - um botão (``KEY``) para representar o sinal ``ready`` do controlador;
 - LEDs para visualizar os sinais ``req`` e ``wEn``;
- ``dram_controller``:
 - Pelo menos **um dos fluxos principais** implementado e testado por simulação:
 - INIT, ou
 - READ, ou

- WRITE, ou
 - REFRESH;
- Pelo menos um dos fluxos restantes em implementação.

Entrega final

Sequência de demonstração

A sequência abaixo deve ser utilizada tanto na demonstração presencial quanto na demonstração por vídeo. Os endereços mencionados são os valores finais que devem ser inseridos nos switches SW[9:4]. Os endereços e valores estão em hexadecimal.

1. Escrever o valor `1` no endereço `0x01`.
2. Escrever o valor `2` no endereço `0x02`.
3. Escrever o valor `3` no endereço `0x0A`.
4. Escrever o valor `4` no endereço `0x40`.
5. Sobrescrever o endereço `0x0A` com o valor `5`.
6. Escrever o valor `6` no endereço `0x4A`.

Após concluir as escritas, realizar as seguintes leituras:

7. Ler o endereço `0x01` → deve retornar `1`.
8. Ler o endereço `0x02` → deve retornar `2`.
9. Ler o endereço `0x0A` → deve retornar `5`.
10. Ler o endereço `0x40` → deve retornar `4`.
11. Ler o endereço `0x4A` → deve retornar `6`.

Durante a demonstração, os integrantes do grupo devem explicar brevemente cada etapa realizada e os resultados observados.

Demonstração por vídeo (1s2026 apenas)

No oferecimento de 2026, está facultada uma demonstração alternativa em vídeo. Os grupos que optarem pela avaliação por vídeo devem gravar uma demonstração curta utilizando a própria placa.

A ideia é que seja um vídeo curto, que pode ser gravado rapidamente com a câmera do celular. Não deve ser realizada nenhuma edição, corte, aceleração, etc.

O vídeo deve mostrar claramente:

- a placa em funcionamento;
- os switches sendo alterados;
- os valores exibidos nos displays;
- a execução completa da sequência de demonstração acima;
- a explicação dos integrantes sobre o que está sendo realizado.

Atenção: A demonstração por vídeo deve ser tratada como uma alternativa excepcional para os grupos que estiverem impedidos de comparecer à demonstração presencial. Sem a interação, o grupo não terá a oportunidade de responder eventuais questionamentos e isso pode impactar negativamente a avaliação.

Simulações

Além da demonstração na placa, espera-se evidência das simulações do controlador DRAM.

Na demonstração presencial, as simulações poderão ser verificadas durante a avaliação e deverão estar incluídas no relatório final.

Espera-se que o relatório apresente explicitamente simulações dos ciclos:

- INIT;
- READ;
- WRITE;
- REFRESH.

As formas de onda devem permitir verificar a sequência de comandos gerados e as principais temporizações utilizadas pelo controlador. Além disso, descrevam em poucas frases por que as formas de onda apresentadas evidenciam o funcionamento correto do fluxo testado, indicando os principais comandos e temporizações observados.

← Rubrica: Demonstração de Planejamento

Jun 3, 2026

Projeto: Controlador VGA →

Mar 22, 2026

© 2026 Isaías Felzmann. All rights reserved.

Made with [Hugo Blox](#).

[Build your site →](#)